ChatApplication Architecture Overview

I. Project structure

```
ChatApplication/
├ ChatApplication.slnx      Solution containing the three projects
├ Program.cs                Launcher for starting Server or Client
├ Common/
  ├ Message.cs              Newline-delimited UTF-8 network I/O
  └── ChatProtocol.cs       Shared commands, channel keys, notifications
├ Server/
  ├── Program.cs            Administration console and server delegate
  ├── ChatServer.cs         TCP listener, sessions, channels, and history
  └── ClientSession.cs      Per-client stream and serialized sending
└── Client/
   ├── Program.cs           Interactive TCP client and receive loop
   └── StockPriceClient.cs  API call for enquiring about a stock's price
```

II. Core architecture

The server owns the TCP listener and keeps connected users in a thread-safe session dictionary. Each ClientSession stores the username, network stream, and current channel. Channel history is maintained by the server and replayed when a client enters broadcast or a private channel.

The Common project is the shared boundary between client and server. Message provides newline-framed UTF-8 transport. ChatProtocol defines JOIN and MESSAGE commands, private-channel keys, and notification formatting/parsing so wire behavior is not duplicated in either application.

III. Asynchronous message handling

Both sides use async/await over NetworkStream. The server awaits AcceptTcpClientAsync, then starts HandleClientAsync for each connection without blocking the accept loop. A client session continuously awaits ReadMessageAsync, parses the command, and routes it to channel switching or message delivery.

Outgoing writes are serialized by a SemaphoreSlim in ClientSession.SendAsync. This matters because broadcasts, history replay, private notifications, and administrative messages may target the same stream concurrently; the lock prevents interleaved bytes on the TCP connection.

When a message is delivered, AddAndSendAsync records it in channel history and asynchronously sends it only to sessions currently on that channel. A private message notification is separately sent to the recipient when that recipient is viewing broadcast, while the private content remains in the private channel.

IV. Delegates and events

Server.Program assigns server.BroadcastAsync to the local MessageSender delegate. This lets the administration command loop invoke the server broadcast operation through a simple callable value while keeping command handling separate from ChatServer implementation.

ChatServer also exposes ActivityLogged as an Action<string> event. Server.Program subscribes with a console logger, so the server reports activity without depending directly on console output. These are lightweight dependency-injection points: the server exposes behavior, while the host decides how to invoke or display it.

V. End-to-end logic

1. Client connects; server requests USERNAME and registers a ClientSession.

2. Client sends JOIN broadcast, JOIN private <user>, MESSAGE <text> or MESSAGE stock <ticker>.

3. Server parses the shared protocol, updates the session channel, or records and routes the message.

4. Each recipient receives complete newline-delimited messages asynchronously; the client recognizes shared notifications and displays banners.